

AmazonHelp AI Support Agent

1. Problem

Build an AI customer-support agent for AmazonHelp that:

1. classifies a customer message into a compact support-intent taxonomy,
2. retrieves historical AmazonHelp interactions that show how similar issues were handled,
3. drafts a grounded response using those historical resolutions,
4. decides whether the request can be handled automatically or should go to a human.

The key design principle is that **retrieval relevance is not the same thing as safe autonomous answering**.

2. Dataset and Golden Set

I selected AmazonHelp because its support corpus contains substantial multi-turn interactions across delivery, orders, refunds, returns, payments, accounts, membership, and digital products.

The evaluation uses a temporal split so evaluation conversations are not present in the retrieval corpus.

The primary Gold evaluation contains **152 genuinely human-labelled examples**.

There are an additional 48 model-assisted examples in the repository, but these are explicitly treated as exploratory and are **not used as primary ground truth**.

3. Intent taxonomy

The final taxonomy contains 11 actionable support intents:

- `delivery\_status`
- `delivery\_problem`
- `order\_issue`
- `preorder\_issue`
- `returns\_replacements`
- `refund\_issue`
- `payment\_billing`
- `account\_security`
- `prime\_membership`
- `digital\_product\_support`
- `general\_support`

Annotation policy assigns the primary intent based on the customer's requested resolution. Escalation is modeled separately from intent.

4. System Architecture

Intent classification

The final intent policy combines high-precision boundary rules with a MiniLM semantic fallback.

This performed better than:

- majority-class prediction,
- TF-IDF + Logistic Regression,

- pure MiniLM centroid classification,
- MiniLM kNN,
- LLM-only few-shot classification.

Historical retrieval

I compared:

- BM25
- dense semantic retrieval
- hybrid BM25 + dense retrieval

The final system uses dense retrieval because stronger human/LLM quality measures favored semantic matches over broad lexical relatedness.

Grounding

Retrieved interactions are evaluated for:

- evidence quality,
- actionability,
- grounding,
- whether autonomous answering is appropriate.

A dense similarity threshold is used only as an engineering pre-filter. It is **not treated as calibrated confidence**.

Response generation

The response generator is implemented in `src/generate.py`.

It receives:

- the customer query,
- retrieved historical AmazonHelp interactions.

The prompt explicitly requires:

- historical evidence only,
- no invented policy,
- no invented refunds or eligibility,
- no unsupported commitments,
- asking for missing information when the evidence is insufficient.

Escalation

The final automation gate is intentionally conservative.

Escalation can occur when:

- the intent is uncertain,
- evidence is weak or insufficient,
- the draft requires additional information,
- the issue is high-risk,
- the evidence does not support a concrete resolution.

5. Evaluation

Intent baseline comparison

The classifier was evaluated against the fixed human-Gold holdout.

Model	Accuracy	Macro F1	Weighted F1
Majority baseline	32.3%	7.0%	15.7%
TF-IDF + Logistic Regression	25.8%	18.9%	24.8%
Final rule + MiniLM	45.2%	35.7%	42.5%

Because several intents are rare or singleton classes, stratified cross-validation would be unstable. A fixed holdout is therefore used for the classifier benchmark.

End-to-end human-Gold evaluation

On the 152-example human-labelled evaluation:

Metric	Result
Intent accuracy	**56.6%**
Intent macro F1	**58.3%**
Intent weighted F1	**56.3%**
Grounded rate	**98.0%**
Strong evidence rate	**36.8%**
High actionability rate	**42.1%**
Auto-answer rate	**3.9%**

Retrieval quality

Dense retrieval achieved:

- 87.0% Useful@5
- 76.0% StrongUseful@5
- 81.0% Actionable@5

under the retrieval quality rubric.

Hybrid achieved a higher broad Useful@5 score, but substantially lower strong usefulness and actionability. Dense retrieval was therefore selected for the final system.

6. What is misleading about my headline number?

The most tempting headline is:

> 98.0% of requests had grounded evidence.

That sounds close to a 98% automation success rate. It is not.

Groundedness only means that the retrieved evidence was related enough for the evaluator to consider the answer supported.

The stricter gates show the difference:

- only ****36.8%**** had strong evidence,
- only ****42.1%**** had high actionability,
- the final conservative system auto-answered only ****3.9%****.

This gap is important. A support system should not turn "something relevant was found" into "answer the customer automatically."

A second misleading number is Useful@5. Hybrid reached 90% Useful@5, but only 23% was StrongUseful@5 and 29% was Actionable@5. Dense retrieval had lower broad usefulness but much stronger results on the stricter measures.

7. Failure Analysis

Failure 1: delivery status vs delivery problem

Queries asking where a package is can contain language associated with delays. The classifier sometimes predicts `delivery_problem` when the actual request is primarily tracking/status.

Hypothesis: lexical overlap around "package", "late", "arrive", and "delivery" creates a difficult boundary.

Next step: collect more human-labelled boundary examples specifically separating status requests from actual delivery failures.

Failure 2: returns vs refunds

Messages mentioning a returned product and money can sit between `returns_replacements` and `refund_issue`.

Hypothesis: customers often describe the whole workflow rather than one atomic support intent.

Next step: annotate based on the desired resolution and add more paired examples from the same workflow.

Failure 3: generic queries

Very short queries such as "Help" can retrieve generally related interactions without enough information for a reliable answer.

Hypothesis: similarity can identify support context without identifying the exact customer problem.

Mitigation: treat insufficiently specified queries as escalation candidates instead of forcing an answer.

Failure 4: multilingual interactions

The original support corpus contains multiple languages.

Hypothesis: semantic retrieval can remain broadly useful while generation becomes less reliable when customer language and retrieved evidence differ.

Next step: measure retrieval and generation separately by language and introduce language-aware prompting/evaluation.

Failure 5: high similarity does not guarantee actionable evidence

A highly similar historical interaction may contain the same topic but not the information required to resolve the customer's specific situation.

Hypothesis: semantic similarity captures topical relatedness more easily than resolution completeness.

Mitigation: separate evidence quality and actionability from retrieval score and require strong evidence before auto-answering.

8. Human Agreement

A separate manual review was performed on a 20-case sample of retrieval/evidence decisions.

The purpose was not to replace the full evaluator, but to test whether automatic judgments agree with a human reviewer on the distinction between useful evidence and evidence safe enough for autonomous handling.

The human review found several cases where evidence was relevant but still insufficient for autonomous handling. This supports the decision to use a conservative automation gate rather than treating retrieval relevance as an answer-confidence score.

9. What I'd do next with one more week

I would prioritize four improvements.

First, expand the human Gold set and deliberately balance rare intents and difficult boundaries.

Second, calibrate the automation gate with more independently-labelled examples rather than relying on retrieval similarity.

Third, evaluate generated responses directly for factuality, unsupported commitments, and policy consistency.

Fourth, build production monitoring around escalation rate, abstention rate, evidence quality, language, and evaluator-human disagreement.

The biggest expected improvement is therefore not a larger model. It is better supervision around the boundary between **relevant evidence** and **safe action**.

10. Reproducibility

The repository uses repository-relative paths rather than Colab-specific paths.

Heavy raw datasets and model weights are not committed.

The response generation layer is API-backed and reads credentials from environment variables.

The repository also contains precomputed evaluation artifacts so the reported experiments do not depend on rerunning expensive evaluation jobs.

See:

- ``README.md``
- ``DECISION_LOG.md``
- ``src/``
- ``results/``
- ``reports/FINAL_REPORT.pdf``